Ho Chi Minh University of Technology

Department of Computer Science and Engineering

CO3057 Computer Vision and Digital Image Processing, *HK252 - 2026*

ArUco Marker Detection Challenge

Project type:	Group (teams of up to 4)
Due:	*Due: 23:55, 01/05/2026*
Submission:	*LMS and Kaggle*
Marks:	*40 - contributes 40% of course grade*

A key application in computer vision is detecting structured visual patterns robustly under real-world variability. Fiducial markers such as **ArUco** [1] are widely used in calibration, robotics, augmented reality, and tracking. However, practical images include motion blur, shadows, low contrast, viewpoint changes, partial occlusions, and cluttered backgrounds. **Your goal in this project is to build a detector that can reliably identify ArUco markers and localize a key corner under such conditions.**

This project is run as a **Kaggle competition** using the **Flying ArUco v2 dataset** [2].

Dataset Overview

The base FlyingArUco v2 dataset consists of images generated by overlaying ArUco markers (and distractor/fake samples) onto backgrounds (sourced from MS COCO 2017 training images).

Figure 1: Example from the Flying ArUco v2 dataset.

Task

Your task is to build a model (or pipeline) which can detect ArUco markers in each image and output:

- the **marker ID**, using `ARUCO_MIP_36h12` dictionary.
- the **top-left corner** (x, y) for each detected marker.

You may use the provided training data (in Kaggle) however you wish to develop your solution. Do not use the datasets from the original website as the values have been modified to prevent probing. Your method may include classical computer vision, learned components, or a hybrid

Figure 2: Example of a detection result with IDs provided

approach, as long as your creativity exists in the code. You need to justify design decisions and discuss failure cases (basically answering all the 'whys'). Just use some existing methods will not get you good marks.

Some example approaches, from basic (low marks) to highly praised (better marks):

- Combine basic image techniques like sharpen, contrast,... to make markers more visible.
- Use feature matching methods with self-elevated tweaks.
- Train Machine Learning models to identify signs.
- Use a CNN model to extract patterns or signs, then make use of them.
- Combination of multiple methods like above.

Example FAQ: Can I use Deep Learning?
Answer: You can use to a limited extent:

- You must not use a bunch of pre-trained models to do all the work for you (e.g. DeepAruco).
- You must be able to explain its working mechanisms and their benefits given your data characteristics.
- You may use pre-trained to aid 1-2 steps (example: Using some CNN to detect presence/quantity of specific objects, then put them as instance attributes).

Kaggle Competition

Evaluation will be performed in a Kaggle-style setting. You will submit a CSV file containing your predictions for each image. Students should register using their university email (domain `@hcmut.edu.vn`).

Notice that there is submission limit to avoid probing. Do not just try and submit until correct. Do not try and create more accounts just for that.

To join in, access this Kaggle Invitation Link.

Submission File Format

Predictions must be submitted as a CSV with columns:

- `image_id`: image filename stem (e.g. `000000000089`)
- `prediction_string`: space-separated triplets `id x y id x y ...`

Example: Marker ID-29 detected at (481.785, 261.833) and Marker ID-102 detected at (273.434, 321.559)

$$\texttt{prediction_string} = \texttt{"29 481.785 261.833 102 273.434 321.559"}$$

Evaluation Metric

Let an image contain a set of ground-truth detections

$$\mathcal{G} = \{(id_i, x_i, y_i)\}_{i=1}^{N_{gt}}$$

and a set of predicted detections

$$\mathcal{P} = \{(\hat{id}_j, \hat{x}_j, \hat{y}_j)\}_{j=1}^{N_{pred}}.$$

Matching Rule

Matching is performed **only by marker ID**. For each marker ID k, define:

$$\mathcal{G}_k = \{(x, y) \in \mathcal{G} \mid id = k\}$$
$$\mathcal{P}_k = \{(\hat{x}, \hat{y}) \in \mathcal{P} \mid \hat{id} = k\}$$

- For each prediction in $\mathcal{P}_k$, compute its minimum Euclidean distance to any ground-truth instance in $\mathcal{G}_k$: $d_{\min} = \min_{(x,y)\in\mathcal{G}_k} \sqrt{(\hat{x} - x)^2 + (\hat{y} - y)^2}$.
- Distances are normalized by the image diagonal: $d_{\text{norm}} = \frac{d_{\min}}{\sqrt{H^2+W^2}}$
- If $|\mathcal{P}_k| > |\mathcal{G}_k|$ (too many guesses for a given ID), only the $|\mathcal{G}_k|$ predictions with the smallest $d_{\min}$ values are considered valid matches.
- The remaining predictions are counted as *spam*.

Distance Score

Each matched prediction contributes a normalized distance score:

$$\phi(d_{\text{norm}}) = \exp\left(-\frac{d_{\text{norm}}^2}{2\sigma^2}\right),$$

where $\sigma = 0.02$ controls tolerance to localization error. This sets the value range to be at $[0.0, 1.0]$.

Per-Image Score

Let N_{gt} denote the total number of ground-truth markers in the image, and let N_{spam} denote the total number of extra predictions (including duplicate or incorrect IDs).

The per-image score is defined as:

$$\text{Score}_{img} = \frac{\sum \phi(d_{\text{norm}})}{N_{gt} + \lambda N_{spam}},$$

where $\lambda = 1$ controls the penalty strength for spam predictions.

Special Case - No Detections: If an image contains no ground-truth markers ($N_{gt} = 0$):

$$\text{Score}_{img} = \begin{cases} 1 & \text{if no predictions are made,} \\ 0 & \text{otherwise.} \end{cases}$$

Final Score

The final leaderboard score is the mean over all images:

$$\text{Final Score} = \frac{1}{M} \sum_{m=1}^{M} \text{Score}_{img}^{(m)}.$$

The metric therefore rewards:

- correct marker ID prediction,
- accurate localization of the top-left corner,
- minimal redundant or incorrect detections.

Group Formation

You should complete this project in a group of up to 4 (minimum 2). You are required to register your group membership via LMS. You may modify your group membership up until the registration deadline. After that point, group membership will be considered final.

Marking Criteria

Component	Marks	Criteria
Wordings and presentation	10	Clarity of writing and organisation; effective use of tables/figures to support results.
Method and justification	10	Correctness of method; motivation and justification of design choices using CV concepts.
Experimentation, evaluation	10	Quality of experiments, evaluation, and error analysis; interpretation of results.
Kaggle performance	10	Meaningful improvement over baseline; stable generalisation.
Team contribution	0	Peer/self assessment and equitable contribution.

Written Report (30/40)

You must submit a written report (PDF). The report should be concise and clearly structured. It should include:

- **Method:** describe your approach and key design decisions.
- **Experiments:** present quantitative results.
- **Evaluation:** discuss robustness under varying conditions.
- **Error analysis:** show failure cases and explain likely causes.

Formatting Requirements

You should follow the structure of Springer LNCS papers (Short paper: 8-11 pages excluding references and appendices) or IEEE (8 pages). You can look for Latex templates on Overleaf (yes, I really recommend using Latex for report typing). Not just following its format, you should look up published paper examples to know how ideas and approaches are presented. Try looking up here, here and here.

Kaggle Performance (10/40)

Ranking/Score will only contribute 10 marks to the total 40 marks of this project. However, you need to surpass the baseline as a hurdle (i.e. get 0 for this project if you cannot make something better than the baseline). This baseline is achieved by just running opencv's built-in AruCo detection function.

10 marks (+2) will be awarded as follows:

- 8 marks for score: $(1 - \frac{0.97 - score}{0.97 - 0.78}) \times 8$
- 2+4 marks for ranking:
  - 0 marks for bottom 10.
  - 2 marks for top 20.
  - 1 mark for the rest.
  - 4 bonus marks for Top 1.
  - 2 bonus marks for Top 2 and 3.
  - 1 bonus mark for Top 4 and 5.

Submission

Submission will be made via *LMS and Kaggle*. Please submit your code and written report separately (unless otherwise instructed). Your submission must include:

- a runnable codebase (`README` required),
- environment specification (e.g. `requirements.txt` / `environment.yml`),
- your Kaggle submission CSV(s),
- and a written report in PDF format.

Do not make your code base public (e.g. GitHub) until final marks are released. Someone will copy you, and I will not tolerate those behaviors.

Late Submissions

-12% from total marks for every delayed day. You need to email with proper reasoning to get deadline extension (no "dog ate my homework" kind of excuse).

Academic Honesty

You may discuss general ideas with other students, but your implementation and report must be your own work. Any inappropriate collaboration, plagiarism, data leakage, or attempts to circumvent competition rules will be treated as academic misconduct under university policy.

Since Kaggle competition performance contributes to the final mark, your submissions to Kaggle must be your own original work and must comply with the competition rules (including submission limits). Submitting predictions that are not produced by your own model(s), or allowing another student to submit your predictions under their account, will be treated as misconduct.

References

[1] S. Garrido-Jurado, R. Muñoz-Salinas, F.J. Madrid-Cuevas, and R. Medina-Carnicer. Generation of fiducial marker dictionaries using mixed integer linear programming. *Pattern Recognition*, 51:481–491, March 2016.

[2] Rafael Berral-Soler, Rafael Muñoz-Salinas, Rafael Medina-Carnicer, and Manuel J. Marín-Jiménez. Deeparuco++: Improved detection of square fiducial markers in challenging lighting conditions. *Image and Vision Computing*, 152:105313, December 2024.